# Sequential Testing for A/B Experiments

A Practical Guide in Three Acts

Version 4

April 2026

## Contents

# Introduction

**The problem.** Every company running A/B tests faces a temptation: peek at the results before the experiment is over. Traditional statistics breaks when you do this — the false positive rate inflates far beyond the promised 5%.

**The solution.** *Sequential testing* provides statistical methods that remain valid no matter when or how often you check. Modern platforms like Eppo build this into their pipeline automatically.

**This guide.** Three acts, each with a simulation, intuitive explanations, and key formulas with plain-language translations:

1. **The Peeking Problem** — Why checking early is dangerous.
2. **The Eppo Solution** — How a modern platform solves it, including the recommended **hybrid approach**.
3. **DIY Alternatives** — What to do if you don't have Eppo, including how to implement the hybrid approach yourself.

**Target audience.** People comfortable with algebra and basic probability. No calculus required. Every formula is accompanied by a plain-language translation.

**What changed from v2:** Added the hybrid approach (sequential testing for guardrail KPIs only, fixed-horizon for the primary KPI) to Act 2 and Act 3, with implementation details and Python code in Appendix A.

# 1 Act 1 — The Peeking Problem

## 1.1 The Motivation

**Intuitive Explanation**

You work at a company running an A/B test. Half your users see the old website, half see a new design. Every day, you check the dashboard: "Is the new design better?" After a week, the dashboard shows $p = 0.03$ — that looks significant! You call it: the new design wins.

But here is the dirty secret of traditional statistics: **the more often you check, the more likely you are to see a false positive.** A test designed to have a 5% false positive rate can easily reach 20–30% if you peek at it repeatedly. One in four or five "winning" experiments might actually be noise.

This is not a theoretical concern. It is the single most common statistical mistake in online experimentation.

## 1.2 The Simulation

**Simulation**

**What is being simulated:** 1,000 A/B experiments where there is *no real effect* — any apparent difference is pure noise.

Two groups of observations (control and treatment) are generated from *identical* distributions. A standard $t$-test is computed after every new observation arrives.

A counter shows: "Number of times $p < 0.05$ so far." The user watches the $p$-value bounce around. Every time it dips below 0.05, it lights up red — a false positive.

**Controls:**

- How many total observations to simulate.
- How many independent experiments to run in parallel.
- Fixed seed toggle (deterministic replay).

A bar chart accumulates: "Fraction of experiments that showed $p < 0.05$ at *some* point during peeking." For 1,000 experiments with regular peeking, this fraction climbs to 20–30%.

A second mode checks only once, at the very end. That fraction stays near 5%.

**Key takeaway:** Peeking inflates false positives. The more you peek, the worse it gets.

## 1.3 Why Does This Happen?

**Intuitive Explanation**

Traditional statistical tests promise: "If there is no real effect, I will falsely cry wolf at most 5% of the time." But this promise comes with fine print: **you may only look at the result once, at a pre-determined time.**
Every time you peek, you get another chance for randomness to fool you. It is like rolling a die repeatedly — the more rolls, the more likely you are to get a six eventually, even though any single roll has only a 1/6 chance.

## 1.4 How Bad Is It?

| Checking schedule | Actual false positive rate |
|---|---|
| Once at the end (as designed) | $\sim$5% |
| Daily for 1 week | $\sim$13% |
| Daily for 2 weeks | $\sim$19% |
| Daily for 4 weeks | $\sim$25% |
| Continuously (every observation) | Can exceed 30% |

If you check a 4-week experiment every day, roughly 1 in 4 "significant" results could be false. This means you might ship features that have no real effect — or even hurt your metrics.

**Key Takeaway**

**The peeking problem:** Checking a traditional test repeatedly inflates the false positive rate far beyond its nominal level. We need a fundamentally different kind of test — one that remains valid no matter when or how often we look.
That kind of test exists. It is called a **sequential test**, and it is what Eppo (and similar platforms) use under the hood.

# 2 Act 2 — The Eppo Solution (Schmit & Miller, 2024)

## 2.1 The Big Idea

**Intuitive Explanation**

Eppo replaces the standard confidence interval with a **sequential confidence interval** — a band around the estimated treatment effect that is valid at *every moment* since the experiment started, not just at one pre-planned analysis time.
The key insight: a sequential CI is wider than a standard CI (you pay a small price for the freedom to peek), but the error guarantee holds no matter when you look. And Eppo reduces the noise in your data using pre-experiment information, making the CI narrower and experiments faster.

## 2.2 The Simulation

**Simulation**

**What is being simulated:** A single A/B experiment flowing through Eppo's pipeline, stage by stage.
**Animated pipeline diagram.** Data flows left-to-right through labelled stages. Each stage lights up as it is explained.
**Stages (rendered as connected boxes):**

$$\boxed{\text{Randomise}} \rightarrow \boxed{\text{Collect}} \rightarrow \boxed{\text{Adjust}} \rightarrow \boxed{\text{Estimate}} \rightarrow \boxed{\text{CI}} \rightarrow \boxed{\text{Decide}}$$

**Step-by-step mode:** A button labelled **"Step through pipeline"** pauses at each stage. At each stage, a distinct visual state is shown:

| Stage | Visual state |
|---|---|
| Randomise | Users appear as dots, randomly coloured blue (control) or green (treatment). |
| Collect | Each dot acquires labels: a pre-experiment value $X$ and an outcome $Y$. Tooltip on hover shows both values. |
| Adjust | Dots animate: $Y \rightarrow Y^*$. A regression line overlays each group. Dots shift vertically to their residual position. Spread visibly decreases. |
| Estimate | Group means appear as horizontal lines. The lift displays as a labelled gap between lines. |
| CI | A confidence interval band draws around the estimated lift. The band narrows as more data arrives. |
| Decide | The CI is colour-coded: **green** (entirely above 0), **red** (entirely below 0), or **grey** (crosses 0). A verdict label appears. |

**Side-by-side comparison mode:** A button labelled **"Compare with / without adjustment"** splits the display into two panels using the *same data* — making the benefit of noise reduction immediately visible.
**Controls:**

- True effect size $\delta$ (0 to 10%).
- Sample size $n$ (100 to 100,000).
- Pre-experiment correlation $\rho$ (0 to 0.95): controls how much noise the adjustment removes.
- Fixed seed toggle (deterministic replay).
- "Run 1,000 experiments" mode (batch statistics).

**Key takeaway:** Each pipeline stage transforms the data in a specific, visible way. Regression adjustment visibly tightens the CI; removing it shows the cost.

## 2.3 The Pipeline, Step by Step

### Step 1: Randomise

Each user arriving at the website is randomly assigned to **control** (existing experience) or **treatment** (new feature). Randomisation ensures the groups are comparable — any difference in outcomes is caused by the treatment, not by pre-existing differences.

### Step 2: Collect data over time

As users interact with the product, two quantities are recorded for each user:

- $Y_i$: the **outcome** during the experiment (e.g. purchase amount, clicks, session length).
- $X_i$: a **pre-experiment covariate** — the same metric from before the experiment started (e.g. purchases in the prior 2 weeks).

### Step 3: Regression adjustment (noise removal)

Users are wildly different — some buy €0, others €500. This noise drowns out the treatment effect. Eppo reduces this noise by using pre-experiment data.

For each group **separately**, fit a model predicting $Y$ from $X$:

$$\hat{f}_{\text{control}}(X), \qquad \hat{f}_{\text{treatment}}(X)$$

Then compute the **adjusted outcome** for each user:

$$Y_i^* = Y_i - \hat{f}_g(X_i)$$

where $g$ is the user's group (control or treatment).

> "Predict what each user would have done based on their history, then subtract that prediction. What's left is the part you *couldn't* have predicted — and that's where the treatment effect hides."
> This is a generalisation of a technique called CUPED (Deng et al., 2013). The stronger the correlation $\rho$ between pre-experiment and experiment behaviour, the more noise is removed: $\text{Var}(Y^*) = \text{Var}(Y)(1-\rho^2)$. At $\rho = 0.7$, half the variance is eliminated — equivalent to doubling your traffic.

**Safety rule:** The covariate $X$ must come from the *pre-experiment* period. Never use data that could be affected by the treatment.

### Step 4: Estimate the treatment effect

Compute the adjusted group means and the **relative lift**:

$$\hat{\tau}(t) = \hat{\mu}_1(t) - \hat{\mu}_0(t), \qquad \text{Relative lift}(t) = \frac{\hat{\tau}(t)}{\hat{\mu}_0(t)}$$

Here $\hat{\mu}_0(t)$ and $\hat{\mu}_1(t)$ are the average adjusted outcomes in the control and treatment groups at time $t$. Their difference $\hat{\tau}(t)$ is the estimated treatment effect. Dividing by the control mean gives a percentage: "the treatment increased purchases by +3%." Relative lift is preferred because it is **scale-invariant** — a 3% lift means the same thing whether the baseline is €10 or €10,000.

**Step 5: Estimate the variance**

$$\hat{\sigma}^2_{\hat{\tau}}(t) = \frac{s_0^2(t)}{n_0(t)} + \frac{s_1^2(t)}{n_1(t)}$$

Here $s_g^2(t)$ is the sample variance (spread) of the adjusted outcomes in group $g$, and $n_g(t)$ is the number of users in that group so far. The variance is estimated from the data — not assumed known.

**Step 6: Construct the sequential confidence interval**

This is the key formula. Instead of a standard CI (which uses the constant $z_{\alpha/2} \approx 1.96$), Eppo uses a time-varying multiplier:

$$\boxed{\mathrm{CI}(t) = \hat{\tau}(t) \ \pm \ \hat{\sigma}_{\hat{\tau}}(t) \cdot \underbrace{\sqrt{\frac{n+\nu}{n} \cdot \log\frac{n+\nu}{\nu\alpha^2}}}_{\text{sequential multiplier}}}$$

**Unpacking the formula:**

- $\hat{\tau}(t)$: the estimated treatment effect at time $t$ (from Step 4).
- $\hat{\sigma}_{\hat{\tau}}(t)$: the estimated standard error (from Step 5).
- $n = n_0(t) + n_1(t)$: total observations so far.
- $\alpha$: significance level (e.g. 0.05 for a 95% CI).
- $\nu$: a tuning parameter set from the expected total sample size $M$: $\nu = M \cdot \hat{\sigma}^2$. It controls *which* sample size the CI is tightest for.
- The "sequential multiplier" replaces the constant 1.96. It starts higher (e.g. $\sim$3.8 early on) and slowly decreases toward 1.96 — but never quite reaches it. This is the **price of peeking**.

This formula comes from the *confidence sequence* framework of Howard et al. (2021), specifically the Normal mixture boundary. The mathematical guarantee: the true treatment effect lies inside the CI at *all times simultaneously* with probability $\geq 1-\alpha$.

**Step 7: Decide**

| What you see | Colour | Action |
|---|---|---|
| CI entirely above 0 | Green | Positive effect. Ship the feature. |
| CI entirely below 0 | Red | Negative effect. Revert. |
| CI crosses 0, experiment ongoing | Grey | Inconclusive. Keep collecting. |
| CI crosses 0, experiment ended | Grey | No significant effect. Decide pragmatically. |

**This decision is valid at whatever time it is made.** No need to pre-specify an analysis time. This solves the peeking problem from Act 1.

**Key Takeaway**

**The Eppo pipeline in one sentence:** Randomise → collect outcomes $Y$ and covariates $X$ → regression-adjust (remove predictable noise) → estimate lift and variance → wrap in a sequential CI → decide when CI excludes zero.

**What makes it work:**

- **Framework:** Howard et al. (2021) confidence sequences.
- **Noise reduction:** Generalised CUPED via per-group regression.
- **Output:** Relative lift with a CI that is valid at any time.

## 2.4 The Hybrid Approach

The full sequential pipeline described above is powerful but comes with a cost: the sequential CI is always wider than a fixed-horizon CI (the "price of peeking"). For the **primary KPI** — the metric you are trying to improve — this wider CI means you need more data to detect the same effect size.

In many organisations, the primary goal of continuous monitoring is not to *declare a winner early* but to **catch damage to guardrail metrics quickly**. This observation leads to a practical hybrid design that combines the best of both worlds.

### The idea

**Intuitive Explanation**

**Use sequential testing where it matters most — early stopping for safety — and fixed-horizon testing where it is most efficient — the primary metric.**

Most experiments are not designed to be stopped early for success. You have a planned duration (say 2–4 weeks), and you want to know at the end whether the treatment improved your primary KPI. What you *do* want is an early warning system: if the treatment is causing real harm to a critical metric (revenue, error rate, latency), you need to know *now*, not in 4 weeks.

The hybrid approach splits your metrics into two categories:

- **Guardrail KPIs** (revenue, error rate, latency, etc.): monitored with a **sequential CI**. You can check these every day or even continuously. If the CI goes fully below zero (or above, for metrics where "up" is bad), abort the experiment immediately.
- **Primary KPI** (the metric you are trying to improve): analysed with a **standard fixed-horizon CI** at the pre-planned end of the experiment. No peeking penalty. Full statistical power.

### Why this works

The peeking problem (Act 1) is a problem of **repeated decision-making on the same metric**. In the hybrid approach:

- The primary KPI is tested **exactly once** (at the end), so there is no multiplicity issue and no need for a wider CI.
- The guardrail KPIs use a sequential CI, which is wider — but this is the right trade-off: you are not trying to detect a subtle improvement in revenue, you are trying to detect *catastrophic damage*. The wider CI is still narrow enough to catch a large negative effect quickly.

### The decision rules

| Metric type | When checked | Decision rule |
|---|---|---|
| Guardrail KPI (e.g. revenue, error rate) | Continuously (sequential CI) | **ABORT** if the sequential CI is entirely on the harmful side of zero at any time during the experiment. |
| Primary KPI (e.g. conversion rate) | Once at the end (fixed-horizon CI) | **SHIP** if the standard CI excludes zero. **NO EFFECT** otherwise. |

### What you gain

| | Full sequential | Hybrid |
|---|---|---|
| Primary KPI CI width | Wider ($\sim$10–40%) | Standard (no penalty) |
| Primary KPI power | Reduced | Full |
| Guardrail protection | Continuous | Continuous |
| Early stopping for success | Yes | No |
| Early stopping for harm | Yes | Yes |
| Complexity | Higher | Lower |

**The trade-off is simple:**

- You give up the ability to stop the experiment early because the primary KPI looks good.
- You gain full statistical power for the primary KPI — your CI is as tight as it can be, so you need fewer users to detect a given effect size.
- You keep the ability to abort early if a guardrail metric is being damaged — the only form of peeking that most teams actually need.

### When to use the hybrid approach

The hybrid approach is the right choice when:

1. You have a **fixed experiment duration** dictated by product cycles, business calendars, or sample-size calculations.
2. You care most about **detecting a small positive effect** on the primary KPI (full power matters).
3. You want a **safety net** for metrics that should never degrade (revenue, latency, error rates).
4. Your team prefers **simple interpretation**: "check the guardrails daily, read the primary result at the end."

The full sequential approach is better when you genuinely want to stop early for success (e.g. a high-risk launch where shipping one week earlier has significant business value).

**Key Takeaway**

**The hybrid approach in one sentence:** Monitor guardrail KPIs with a sequential CI for early abort; analyse the primary KPI with a standard CI at the planned end-date.
This gives you the safety of sequential testing where it matters most (preventing harm) without paying the power penalty on the metric you care about most (the primary KPI).
**This is the recommended approach for most A/B testing programmes.**

# 3 Act 3 — What If You Don't Have Eppo?

Act 2 described Eppo's sophisticated pipeline. But many teams run experiments without a dedicated platform. This act presents three progressively better approaches to peek-safe testing that anyone can implement with a spreadsheet or a few lines of code — and compares them head-to-head with Eppo. It also shows how to implement the hybrid approach from scratch.

## 3.1 The Simulation

**Simulation**

**What is being simulated:** A single A/B experiment monitored at $K$ pre-planned analysis times, showing four methods side-by-side.

**Panel 1 — Four confidence bands:** The same experiment, with four CI bands evolving over time:

- **Red (Bonferroni):** Widest. Simple but conservative.
- **Orange (Pocock):** Constant threshold, tighter than Bonferroni.
- **Blue (O'Brien–Fleming):** Starts very wide, narrows to near-classical at the final analysis.
- **Green (Eppo / Howard):** Continuous monitoring. No need to pre-specify $K$.

Vertical dashed lines mark the $K$ planned analysis times. Between analyses, the group-sequential methods have no valid CI — only Eppo's band is defined everywhere.

**Panel 2 — 10,000 experiments:** Run 10,000 experiments under the null ($\delta = 0$). Counters show the false positive rate for each method:

- Classical CI with $K$ peeks: $\gg 5\%$.
- Bonferroni: $\leq 5\%$ (often well below).
- Pocock: $\leq 5\%$ (closer to 5%).
- O'Brien–Fleming: $\leq 5\%$.
- Eppo: $\leq 5\%$.

**Panel 3 — Efficiency comparison:** Under a true effect ($\delta > 0$), show average stopping time for each method.

**Controls:**

- Number of planned peeks $K$ (2 to 20).
- True effect size $\delta$ (0 to 10%).
- Significance level $\alpha$.
- Fixed seed toggle (deterministic replay).
- "Run 1,000 experiments" mode.

**Key takeaway:** All four methods control the false positive rate. The price of simplicity is wider intervals (slower detection). Even the simplest method (Bonferroni) is far better than naive peeking.

## 3.2 Intuitive Explanation

**Intuitive Explanation**

You want to check your experiment every Monday. If you use a standard 95% CI each time, you'll get false positives far too often (Act 1).
The simplest fix: **be stricter at each check.** If you plan to peek 4 times, demand stronger evidence each time. The three methods below differ in *how* they distribute that strictness across the peeks.
Think of it as a budget. You have a total error budget of $\alpha = 5\%$.

- **Bonferroni** distributes the budget equally: $\alpha/K$ per peek.
- **Pocock** distributes it more cleverly (using the correlation between test statistics at different peeks), but still equally across peeks.
- **O'Brien–Fleming** front-loads the budget: almost no spending early, most of it saved for the final analysis.

Eppo's approach (Act 2) doesn't need a fixed budget at all — it works for *any* number of peeks, continuously.

## 3.3 Method 1: Bonferroni Correction (Simplest)

### The idea

Before the experiment starts, commit to checking exactly $K$ times (e.g. every Monday for 4 weeks: $K = 4$). At each check, use a significance level of $\alpha/K$ instead of $\alpha$.

### The recipe

1. Choose $K$ (number of planned peeks) and $\alpha$ (e.g. 0.05).
2. At each peek $k = 1, \ldots, K$, compute the standard CI using the adjusted significance level:
$$\text{CI}_k = \hat{\tau}(t_k) \ \pm \ \hat{\sigma}_{\hat{\tau}}(t_k) \cdot z_{\alpha/(2K)}$$
where $\hat{\tau}(t_k)$ is the estimated treatment effect at peek $k$, $\hat{\sigma}_{\hat{\tau}}(t_k)$ is its standard error, and $z_{\alpha/(2K)}$ is the critical value from a Normal distribution at the adjusted significance level.
3. If the CI does not cross 0: stop and declare significance.
4. If $k = K$ and the CI crosses 0: no significant effect.

| $K$ (peeks) | $\alpha/K$ | $z_{\alpha/(2K)}$ | CI multiplier vs. 1.96 |
|---|---|---|---|
| 1 | 0.050 | 1.96 | 1.00× (no correction) |
| 2 | 0.025 | 2.24 | 1.14× |
| 4 | 0.0125 | 2.50 | 1.28× |
| 8 | 0.00625 | 2.73 | 1.39× |
| 12 | 0.00417 | 2.87 | 1.46× |
| 20 | 0.0025 | 3.02 | 1.54× |
| 52 | 0.00096 | 3.29 | 1.68× (weekly for a year) |

With 4 planned peeks, replace $z = 1.96$ with $z = 2.50$. Your CI is 28% wider at each peek, but you can safely check 4 times without inflating errors.

### Implementation step by step

Given data arrays `control[]` and `treatment[]` at each scheduled peek:

1. **Compute group statistics:**

$$\bar{Y}_0 = \frac{1}{n_0}\sum_{i=1}^{n_0} Y_{0,i}, \qquad s_0^2 = \frac{1}{n_0 - 1}\sum_{i=1}^{n_0}(Y_{0,i} - \bar{Y}_0)^2$$

   and analogously for group 1.

2. **Compute the estimated treatment effect and its standard error:**

$$\hat{\tau} = \bar{Y}_1 - \bar{Y}_0, \qquad \text{SE} = \sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}}$$

3. **Look up the adjusted critical value:**

$$z^* = z_{\alpha/(2K)} = \Phi^{-1}\left(1 - \tfrac{\alpha}{2K}\right)$$

   where $\Phi^{-1}$ is the inverse of the standard Normal CDF (e.g. `scipy.stats.norm.ppf` in Python).

4. **Construct the CI and decide:**

$$\text{CI} = \left[\hat{\tau} - z^* \cdot \text{SE},\ \hat{\tau} + z^* \cdot \text{SE}\right]$$

   If the CI does not contain 0, stop and declare significance. Otherwise, wait until the next scheduled peek.

### Pros and cons

+ **Dead simple.** One line of code: change `1.96` to `z_alpha/(2K)`.
+ **Always valid.** Works for any test statistic, any distribution.
− **Conservative.** The CI is wider than necessary because Bonferroni ignores the correlation between test statistics at successive peeks.
− **Scales poorly.** As $K$ grows, the correction becomes increasingly harsh.

## 3.4 Method 2: Pocock Boundaries (1977)

### The idea

Like Bonferroni, pre-specify $K$ equally-spaced analysis times. But instead of Bonferroni's crude $\alpha/K$ split, use a constant critical value $c_P$ that accounts for the correlation between test statistics at different peeks.

Because the test statistics $Z_1, Z_2, \ldots, Z_K$ at successive analyses share overlapping data, they are positively correlated. Pocock's method exploits this to derive a tighter threshold.

### The recipe

1. Choose $K$ and $\alpha$.
2. Look up (or compute) the Pocock critical value $c_P(K, \alpha)$.
3. At each peek $k$, compute the $z$-statistic: $Z_k = \hat{\tau}(t_k)/\hat{\sigma}_{\hat{\tau}}(t_k)$.
4. If $|Z_k| > c_P$: stop and declare significance.

| $K$ | $c_P$ (Pocock) | $z_{\alpha/(2K)}$ (Bonf.) | Improvement |
|---|---|---|---|
| 2 | 2.18 | 2.24 | 3% tighter |
| 4 | 2.36 | 2.50 | 6% tighter |
| 8 | 2.51 | 2.73 | 8% tighter |
| 12 | 2.60 | 2.87 | 9% tighter |
| 20 | 2.69 | 3.02 | 11% tighter |

Pocock gives a constant threshold that is a few percent lower than Bonferroni's. The key advantage: the threshold is the same at every peek, so it is almost as simple to use as Bonferroni.

### Implementation step by step

The Pocock critical value $c_P$ is found by solving the following equation numerically. Let $\mathbf{Z} = (Z_1, \ldots, Z_K)$ be the vector of $z$-statistics at the $K$ equally-spaced analyses. Under the null, these follow a multivariate Normal distribution with correlation matrix:

$$\mathrm{Cor}(Z_j, Z_k) = \sqrt{\frac{\min(j,k)}{\max(j,k)}} \qquad \text{for } 1 \le j, k \le K$$

The Pocock boundary $c_P$ is the unique value satisfying:

$$\mathbb{P}\Big(\max_{1 \le k \le K} |Z_k| > c_P\Big) = \alpha$$

This is computed via multivariate Normal integration (see Appendix A for Python code using `scipy.stats.multivariate_normal`).

**Once $c_P$ is known**, the decision rule at each peek is identical to Bonferroni, but with $c_P$ replacing $z_{\alpha/(2K)}$:

1. At peek $k$, compute:
$$Z_k = \frac{\hat{\tau}(t_k)}{\mathrm{SE}(t_k)}$$
2. If $|Z_k| > c_P$: reject the null. Otherwise: continue.

## 3.5 Method 3: O'Brien–Fleming Boundaries (1979)

### The idea

Instead of a constant threshold, use a threshold that *decreases* over time. At the first peek, demand very strong evidence; at the last peek, demand nearly the same evidence

as a classical test.

### The recipe

At peek $k$ (of $K$ total), use the critical value:

$$c_k^{\text{OBF}} \approx z_{\alpha/2} \cdot \sqrt{K/k}$$

where $z_{\alpha/2} \approx 1.96$ is the standard Normal critical value.

| Peek $k$ (of $K$=4) | $c_k^{\text{OBF}}$ | $c_P$ (Pocock) | $z_{\alpha/(2K)}$ (Bonf.) | $z_{\alpha/2}$ (classical) |
|---|---|---|---|---|
| 1 (25% of data) | 4.05 | 2.36 | 2.50 | 1.96 |
| 2 (50% of data) | 2.86 | 2.36 | 2.50 | 1.96 |
| 3 (75% of data) | 2.34 | 2.36 | 2.50 | 1.96 |
| 4 (100% of data) | 2.02 | 2.36 | 2.50 | 1.96 |

O'Brien–Fleming is extremely conservative early (threshold 4.05 at the first peek — you'd almost never stop that early) but barely penalises the final analysis (2.02 vs. 1.96 — only 3% wider).
This is the most efficient approach: if you run the experiment to completion, you pay almost nothing for the option to peek.

### Implementation step by step

The O'Brien–Fleming boundary is the simplest to compute of the three "non-trivial" methods — it has a closed-form approximation.

1. **Before the experiment:** Fix $K$ and $\alpha$. At each peek $k$, the fraction of total data collected is $t_k = k/K$ (assuming equal spacing).
2. **At peek $k$, compute the critical value:**

   $$c_k^{\text{OBF}} = \frac{z_{\alpha/2}}{\sqrt{t_k}} = z_{\alpha/2} \cdot \sqrt{\frac{K}{k}}$$

   This is the Lan–DeMets spending function approximation to the original O'Brien–Fleming boundary.
3. **Compute the $z$-statistic** (same as Bonferroni and Pocock):

   $$Z_k = \frac{\hat{\tau}(t_k)}{\text{SE}(t_k)}$$

4. **Decide:** If $|Z_k| > c_k^{\text{OBF}}$: reject. Otherwise: continue to peek $k+1$.
5. **Convert to a confidence interval** (optional but recommended for interpretation):

   $$\text{CI}_k = \hat{\tau}(t_k) \pm c_k^{\text{OBF}} \cdot \text{SE}(t_k)$$

   Note: the CI width *changes* across peeks because $c_k^{\text{OBF}}$ varies.

**Practical tip:** The exact O'Brien–Fleming boundaries (computed via multivariate Normal integration, like Pocock) differ slightly from the $z_{\alpha/2}/\sqrt{t_k}$ approximation. The approximation is accurate for $K \leq 10$ and is what most software implements. For larger $K$, use a library (e.g. `statsmodels.stats.multitest` or the R package `gsDesign`).

## 3.6 Head-to-Head Comparison

| | **Bonferroni** | **Pocock** | **OBF** | **Eppo** |
|---|---|---|---|---|
| Pre-specify $K$? | Yes | Yes | Yes | No |
| Threshold type | Constant | Constant | Decreasing | Continuous |
| CI at peek 1 ($K$=4) | $2.50 \times \hat{\sigma}$ | $2.36 \times \hat{\sigma}$ | $4.05 \times \hat{\sigma}$ | $\sim 3.8 \times \hat{\sigma}$ |
| CI at final analysis ($K$=4) | $2.50 \times \hat{\sigma}$ | $2.36 \times \hat{\sigma}$ | $2.02 \times \hat{\sigma}$ | $\sim 2.3 \times \hat{\sigma}$ |
| Valid between peeks? | No | No | No | Yes |
| Variance reduction? | Manual | Manual | Manual | Built-in |
| Implementation | 1 line | Lookup table | Formula | Platform |

**Key Takeaway**

**Which method should you use?**

- **No platform, minimal effort:** Bonferroni. Commit to $K$ peeks. Replace 1.96 with $z_{\alpha/(2K)}$. Done.
- **No platform, moderate effort:** O'Brien–Fleming. Nearly as efficient as Eppo at the final analysis, but requires pre-specifying $K$ and cannot peek between scheduled analyses.
- **Need continuous monitoring or variance reduction:** Use a platform like Eppo. Continuous validity, regression adjustment, and no need to pre-commit to $K$.

**The bottom line:** even the crudest correction (Bonferroni) is *vastly* better than naive peeking. If you are checking results without any correction, you should start with Bonferroni *today*.

## 3.7 Implementing the Hybrid Approach Without Eppo

Act 2 described the hybrid approach: sequential CI on guardrail KPIs for early abort, standard CI on the primary KPI at the end. Here is how to implement it without a platform, using any of the three methods above.

### Step 1: Classify your metrics

Before the experiment starts, split your metrics into two lists:

| Category | Examples | Testing method |
|---|---|---|
| Primary KPI | Conversion rate, sign-ups | Fixed-horizon ($z = 1.96$) |
| Guardrail KPIs | Revenue, error rate, latency, crash rate | Sequential (any of the 3 methods) |

### Step 2: Choose your guardrail monitoring schedule

Decide how often you will check guardrail KPIs:

- **Weekly** (recommended for most teams): $K$ = number of weeks in the experiment.
- **Daily**: $K$ = number of days. Use O'Brien–Fleming to avoid the harsh Bonferroni penalty at large $K$.

### Step 3: Set guardrail significance levels

You are testing *multiple* guardrail KPIs, each with $K$ peeks. To control the overall false positive rate across all guardrails:

**Option A (simplest):** Use a Bonferroni correction *across guardrails*. If you have $G$ guardrail KPIs, set $\alpha_{\text{guardrail}} = \alpha/G$ for each one.

**Example:** 3 guardrails ($G = 3$), $\alpha = 0.05$, 4 weekly peeks ($K = 4$):

$$\alpha_{\text{per guardrail}} = \frac{0.05}{3} \approx 0.0167$$

Then apply Bonferroni within each guardrail:

$$z^* = \Phi^{-1}\left(1 - \tfrac{0.0167}{2\times 4}\right) = \Phi^{-1}(0.9979) \approx 2.86$$

**Option B (tighter):** Use O'Brien–Fleming within each guardrail, with $\alpha_{\text{per guardrail}} = \alpha/G$. The final-peek threshold will be close to $z_{\alpha/(2G)} \approx 2.39$ (for $G = 3$), which is much less wasteful than Bonferroni's constant 2.86.

### Step 4: Run the experiment

At each scheduled peek:

1. For **each guardrail KPI**: compute the CI using your chosen sequential method (Bonferroni, Pocock, or O'Brien–Fleming) with the per-guardrail $\alpha$.
2. If **any** guardrail CI is entirely on the harmful side of zero: **ABORT** the experiment.
3. Otherwise: continue.

At the **end** of the experiment:

1. For the **primary KPI**: compute a standard CI with the full $\alpha$ (or your preferred primary $\alpha$):
$$\text{CI}_{\text{primary}} = \hat{\tau} \ \pm \ z_{\alpha/2} \cdot \text{SE}$$
2. **SHIP** if the CI excludes zero. Otherwise: no significant effect.

**Summary: the complete recipe**

| When | What to do |
|---|---|
| Before experiment | Classify metrics (primary vs. guardrail). Choose $K$, $\alpha$, $G$. Compute per-guardrail $\alpha_g = \alpha/G$ and critical values for your chosen method. |
| At each peek $k$ | For each guardrail: compute $Z_k$ and compare against the sequential threshold. Abort if any guardrail is significantly harmed. |
| At scheduled end | Compute the primary KPI CI with $z_{0.025} = 1.96$ (no penalty). Ship or don't ship based on this single test. |

**Key Takeaway**

**The hybrid approach without Eppo:**

- Apply Bonferroni across your $G$ guardrail KPIs: $\alpha_g = \alpha/G$.
- Within each guardrail, use O'Brien–Fleming (best) or Bonferroni (simplest) across $K$ peeks.
- Test the primary KPI once at the end with a standard CI — no correction needed.
- This is the **recommended approach** for teams without a sequential testing platform.

# 4 Summary

| Act | Topic | Key message |
|---|---|---|
| 1 | The Peeking Problem | Checking a standard test repeatedly inflates false positives to 20–30%. You need a different kind of test. |
| 2 | The Eppo Solution | A sequential CI is valid at every moment. The **hybrid approach** uses sequential testing only for guardrail KPIs (early abort) and a standard CI for the primary KPI (full power). |
| 3 | DIY Alternatives | Bonferroni (simplest), Pocock, O'Brien–Fleming — three ways to peek safely without a platform. The hybrid approach works with any of them. |

**The core trade-off:** if you want to peek, you must pay — your confidence interval must be wider. The question is how much wider, and the answer depends on how sophisticated a method you use:

| Method | Price of peeking |
|---|---|
| No correction (naive) | Wrong answers (20–30% false positives) |
| Bonferroni | CI $\sim$28–68% wider (depending on $K$) |
| Pocock | CI $\sim$20–37% wider |
| O'Brien–Fleming | CI $\sim$3% wider at final analysis |
| Eppo (sequential CI) | CI $\sim$10–40% wider, valid *continuously* |
| Hybrid (recommended) | **No penalty on primary KPI**; sequential penalty on guardrails only |

# 5 References


1. **Deng, A., Xu, Y., Kohavi, R., & Walker, T.** (2013). Improving the sensitivity of online controlled experiments by utilizing pre-experiment data. *Proc. WSDM*, 123–132. `https://doi.org/10.1145/2433396.2433413` — The original CUPED paper. Variance reduction via pre-experiment covariates.

2. **Howard, S. R., Ramdas, A., McAuliffe, J., & Sekhon, J.** (2021). Time-uniform, nonparametric, nonasymptotic confidence sequences. *Annals of Statistics*, 49(2), 1055–1080. `https://doi.org/10.1214/20-AOS1991` — The confidence sequence framework that Eppo adopted.

3. **Pocock, S. J.** (1977). Group sequential methods in the design and analysis of clinical trials. *Biometrika*, 64(2), 191–199.

4. **O'Brien, P. C. & Fleming, T. R.** (1979). A multiple testing procedure for clinical trials. *Biometrics*, 35(3), 549–556.

5. **Schmit, M. & Miller, T.** (2024). Sequential confidence intervals for comparing two proportions with variance reduction. *Eppo Technical Report*. — Eppo's implementation combining Howard et al.'s confidence sequences with generalised CUPED.

# A Python Code

This appendix provides self-contained Python code for:

1. A simulation demonstrating the peeking problem (Act 1).
2. Implementation snippets for each DIY method (Act 3).
3. A hybrid approach implementation (Act 2 & Act 3).

All code requires `numpy` and `scipy`. Install with: `pip install numpy scipy`.

## A.1 Simulation: The Peeking Problem

This simulation runs many A/B experiments under the null (no true effect) and measures how often naive peeking produces a false positive, versus checking only at the end.

```python
import numpy as np
from scipy import stats

def simulate_peeking(
    n_experiments: int = 10_000,
    n_obs_per_group: int = 500,
    peek_interval: int = 10,
    alpha: float = 0.05,
    seed: int = 42,
) -> dict:
    """Simulate the peeking problem under the null hypothesis.

    Returns a dict with false positive rates for:
      - 'naive_peeking': check every peek_interval observations
      - 'single_check':  check only at the end
    """
    rng = np.random.default_rng(seed)
    peek_times = list(range(peek_interval,
                            n_obs_per_group + 1,
                            peek_interval))

    false_pos_peek = 0
    false_pos_end = 0

    for _ in range(n_experiments):
        # Generate data from identical distributions (null)
        control = rng.normal(0, 1, n_obs_per_group)
        treatment = rng.normal(0, 1, n_obs_per_group)

        # --- Naive peeking ---
        ever_significant = False
        for t in peek_times:
            ctrl_t = control[:t]
            trt_t = treatment[:t]
            _, p_value = stats.ttest_ind(ctrl_t, trt_t)
            if p_value < alpha:
                ever_significant = True
```

```
                break  # stop at first significance
        if ever_significant:
            false_pos_peek += 1

        # --- Single check at the end ---
        _, p_value_end = stats.ttest_ind(control, treatment)
        if p_value_end < alpha:
            false_pos_end += 1

    return {
        "naive_peeking": false_pos_peek / n_experiments,
        "single_check": false_pos_end / n_experiments,
        "n_experiments": n_experiments,
        "n_peeks": len(peek_times),
    }

# --- Run it ---
results = simulate_peeking()
print(f"False positive rate (single check): "
      f"{results['single_check']:.1%}")
print(f"False positive rate (peeking every "
      f"{10} obs, {results['n_peeks']} peeks): "
      f"{results['naive_peeking']:.1%}")
```

Expected output (approximately):

```
False positive rate (single check): 5.0%
False positive rate (peeking every 10 obs, 50 peeks): 27.3%
```

## A.2 Implementation: Bonferroni Correction

```
import numpy as np
from scipy import stats

def bonferroni_test(
    control: np.ndarray,
    treatment: np.ndarray,
    K: int,
    alpha: float = 0.05,
) -> dict:
    """Run a single-peek Bonferroni-corrected test.

    Call this function at each of the K scheduled peeks
    with the data collected so far.

    Returns:
      - 'reject': whether to reject the null
      - 'ci': the confidence interval for tau_hat
      - 'tau_hat': estimated treatment effect
      - 'z_star': adjusted critical value
```

```python
    """
    n0, n1 = len(control), len(treatment)
    tau_hat = treatment.mean() - control.mean()
    se = np.sqrt(control.var(ddof=1) / n0
                 + treatment.var(ddof=1) / n1)

    # Bonferroni-adjusted critical value
    z_star = stats.norm.ppf(1 - alpha / (2 * K))

    ci_lower = tau_hat - z_star * se
    ci_upper = tau_hat + z_star * se
    reject = ci_lower > 0 or ci_upper < 0

    return {
        "reject": reject,
        "ci": (ci_lower, ci_upper),
        "tau_hat": tau_hat,
        "z_star": z_star,
        "se": se,
    }

# --- Example usage ---
rng = np.random.default_rng(0)
K = 4  # plan to peek 4 times
N_total = 2000  # total obs per group
peek_every = N_total // K

control_all = rng.normal(10, 3, N_total)
treatment_all = rng.normal(10.3, 3, N_total)  # small effect

for k in range(1, K + 1):
    n = k * peek_every
    result = bonferroni_test(
        control_all[:n], treatment_all[:n], K=K
    )
    ci = result["ci"]
    status = "REJECT" if result["reject"] else "continue"
    print(f"Peek {k}/{K} (n={n}): "
          f"tau={result['tau_hat']:.3f}, "
          f"CI=[{ci[0]:.3f}, {ci[1]:.3f}], "
          f"z*={result['z_star']:.3f} -> {status}")
```

## A.3 Implementation: Pocock Boundaries

```python
import numpy as np
from scipy import stats, optimize

def pocock_critical_value(
    K: int, alpha: float = 0.05, n_sim: int = 200_000,
    seed: int = 42,
```

```python
) -> float:
    """Find the Pocock boundary c_P by simulation.

    Simulates the multivariate Normal distribution of
    (Z_1, ..., Z_K) under the null and finds c_P such that
    P(max |Z_k| > c_P) = alpha.

    For production use, replace with exact multivariate
    Normal integration (e.g. mvnorm from R or PyMVPA).
    """
    rng = np.random.default_rng(seed)

    # Build the correlation matrix: Cor(Z_j, Z_k) = sqrt(min/max)
    fracs = np.arange(1, K + 1) / K
    corr = np.zeros((K, K))
    for j in range(K):
        for k in range(K):
            corr[j, k] = np.sqrt(
                min(fracs[j], fracs[k])
                / max(fracs[j], fracs[k])
            )

    # Simulate Z vectors
    samples = rng.multivariate_normal(
        np.zeros(K), corr, size=n_sim
    )
    max_abs_z = np.abs(samples).max(axis=1)

    # Find c_P: the (1 - alpha) quantile of max|Z|
    c_P = np.quantile(max_abs_z, 1 - alpha)
    return float(c_P)

def pocock_test(
    control: np.ndarray,
    treatment: np.ndarray,
    c_P: float,
) -> dict:
    """Test at a single peek using the Pocock boundary."""
    n0, n1 = len(control), len(treatment)
    tau_hat = treatment.mean() - control.mean()
    se = np.sqrt(control.var(ddof=1) / n0
                 + treatment.var(ddof=1) / n1)
    z_stat = tau_hat / se
    reject = abs(z_stat) > c_P

    ci_lower = tau_hat - c_P * se
    ci_upper = tau_hat + c_P * se

    return {
        "reject": reject,
```

```
        "z_stat": z_stat,
        "ci": (ci_lower, ci_upper),
        "tau_hat": tau_hat,
    }

# --- Example usage ---
K = 4
c_P = pocock_critical_value(K)
print(f"Pocock critical value for K={K}: {c_P:.3f}")

rng = np.random.default_rng(0)
N_total = 2000
peek_every = N_total // K

control_all = rng.normal(10, 3, N_total)
treatment_all = rng.normal(10.3, 3, N_total)

for k in range(1, K + 1):
    n = k * peek_every
    result = pocock_test(
        control_all[:n], treatment_all[:n], c_P=c_P
    )
    ci = result["ci"]
    status = "REJECT" if result["reject"] else "continue"
    print(f"Peek {k}/{K} (n={n}): "
          f"Z={result['z_stat']:.3f}, "
          f"CI=[{ci[0]:.3f}, {ci[1]:.3f}] -> {status}")
```

## A.4 Implementation: O'Brien–Fleming Boundaries

```
import numpy as np
from scipy import stats

def obf_critical_value(k: int, K: int,
                       alpha: float = 0.05) -> float:
    """O'Brien-Fleming critical value at peek k of K.

    Uses the Lan-DeMets spending function approximation:
        c_k = z_{alpha/2} / sqrt(k / K)
    """
    z_alpha2 = stats.norm.ppf(1 - alpha / 2)
    t_k = k / K  # information fraction
    return z_alpha2 / np.sqrt(t_k)

def obf_test(
    control: np.ndarray,
    treatment: np.ndarray,
    k: int,
    K: int,
    alpha: float = 0.05,
```

```python
) -> dict:
    """Test at peek k of K using O'Brien-Fleming."""
    n0, n1 = len(control), len(treatment)
    tau_hat = treatment.mean() - control.mean()
    se = np.sqrt(control.var(ddof=1) / n0
                 + treatment.var(ddof=1) / n1)
    z_stat = tau_hat / se
    c_k = obf_critical_value(k, K, alpha)
    reject = abs(z_stat) > c_k

    ci_lower = tau_hat - c_k * se
    ci_upper = tau_hat + c_k * se

    return {
        "reject": reject,
        "z_stat": z_stat,
        "c_k": c_k,
        "ci": (ci_lower, ci_upper),
        "tau_hat": tau_hat,
    }

# --- Example usage ---
K = 4
rng = np.random.default_rng(0)
N_total = 2000
peek_every = N_total // K

control_all = rng.normal(10, 3, N_total)
treatment_all = rng.normal(10.3, 3, N_total)

for k in range(1, K + 1):
    n = k * peek_every
    result = obf_test(
        control_all[:n], treatment_all[:n], k=k, K=K
    )
    ci = result["ci"]
    status = "REJECT" if result["reject"] else "continue"
    print(f"Peek {k}/{K} (n={n}): "
          f"Z={result['z_stat']:.3f}, "
          f"c_k={result['c_k']:.3f}, "
          f"CI=[{ci[0]:.3f}, {ci[1]:.3f}] -> {status}")
```

## A.5 Implementation: Hybrid Approach

This snippet implements the complete hybrid workflow: O'Brien–Fleming on guardrail KPIs at each peek, standard CI on the primary KPI at the end.

```python
import numpy as np
from scipy import stats

```

```python
def hybrid_experiment(
    primary_ctrl: np.ndarray,
    primary_trt: np.ndarray,
    guardrail_data: dict[str, tuple[np.ndarray, np.ndarray]],
    K: int = 4,
    alpha: float = 0.05,
    guardrail_direction: dict[str, str] | None = None,
) -> dict:
    """Run a hybrid sequential/fixed-horizon experiment.

    Args:
        primary_ctrl, primary_trt:
            Full arrays for the primary KPI.
        guardrail_data:
            Dict mapping guardrail name -> (ctrl_array, trt_array).
        K: Number of planned peeks for guardrails.
        alpha: Overall significance level.
        guardrail_direction:
            Dict mapping guardrail name -> 'lower' or 'upper'.
            'lower' means harm = treatment < control (default).
            'upper' means harm = treatment > control
            (e.g. error_rate).

    Returns:
        Dict with 'aborted', 'abort_reason', 'abort_peek',
        'primary_result'.
    """
    if guardrail_direction is None:
        guardrail_direction = {}

    G = len(guardrail_data)
    alpha_per_guardrail = alpha / G if G > 0 else alpha
    n_total = len(primary_ctrl)
    peek_every = n_total // K

    # --- Phase 1: Monitor guardrails sequentially ---
    for k in range(1, K + 1):
        n = k * peek_every

        for name, (g_ctrl, g_trt) in guardrail_data.items():
            c, t = g_ctrl[:n], g_trt[:n]
            tau = t.mean() - c.mean()
            se = np.sqrt(
                c.var(ddof=1) / len(c)
                + t.var(ddof=1) / len(t)
            )
            # OBF critical value with per-guardrail alpha
            c_k = obf_critical_value(
                k, K, alpha_per_guardrail
            )
```

```python

            direction = guardrail_direction.get(
                name, "lower"
            )
            # Check for harm only (one-sided)
            if direction == "lower" and tau / se < -c_k:
                return {
                    "aborted": True,
                    "abort_reason": f"{name} harmed",
                    "abort_peek": k,
                    "guardrail_z": tau / se,
                    "guardrail_threshold": -c_k,
                    "primary_result": None,
                }
            if direction == "upper" and tau / se > c_k:
                return {
                    "aborted": True,
                    "abort_reason": f"{name} harmed",
                    "abort_peek": k,
                    "guardrail_z": tau / se,
                    "guardrail_threshold": c_k,
                    "primary_result": None,
                }

    # --- Phase 2: Analyse primary KPI at end ---
    tau = primary_trt.mean() - primary_ctrl.mean()
    se = np.sqrt(
        primary_ctrl.var(ddof=1) / len(primary_ctrl)
        + primary_trt.var(ddof=1) / len(primary_trt)
    )
    z = stats.norm.ppf(1 - alpha / 2)
    ci = (tau - z * se, tau + z * se)

    return {
        "aborted": False,
        "abort_reason": None,
        "abort_peek": None,
        "primary_result": {
            "tau_hat": tau,
            "se": se,
            "ci": ci,
            "significant": ci[0] > 0 or ci[1] < 0,
        },
    }


# --- Example: 4-week experiment, 3 guardrails ---
rng = np.random.default_rng(42)
N = 4000

# Primary KPI: small positive effect
```

```python
primary_ctrl = rng.normal(100, 30, N)
primary_trt = rng.normal(101, 30, N)

# Guardrails: revenue (no harm), error rate (no harm),
#             latency (treatment causes harm!)
guardrails = {
    "revenue": (
        rng.normal(50, 15, N),      # control
        rng.normal(50, 15, N),      # treatment (no effect)
    ),
    "error_rate": (
        rng.normal(0.02, 0.01, N),
        rng.normal(0.02, 0.01, N),  # no effect
    ),
    "latency_ms": (
        rng.normal(200, 50, N),
        rng.normal(220, 50, N),     # +20ms harm!
    ),
}

result = hybrid_experiment(
    primary_ctrl, primary_trt,
    guardrails,
    K=4,
    guardrail_direction={
        "revenue": "lower",        # harm = decrease
        "error_rate": "upper",     # harm = increase
        "latency_ms": "upper",     # harm = increase
    },
)

if result["aborted"]:
    print(f"ABORTED at peek {result['abort_peek']}: "
          f"{result['abort_reason']}")
    print(f"  Z = {result['guardrail_z']:.3f}, "
          f"threshold = {result['guardrail_threshold']:.3f}")
else:
    pr = result["primary_result"]
    sig = "SIGNIFICANT" if pr["significant"] else "not
        significant"
    print(f"Experiment completed. Primary KPI: {sig}")
    print(f"  tau = {pr['tau_hat']:.3f}, "
          f"CI = [{pr['ci'][0]:.3f}, {pr['ci'][1]:.3f}]")
```

Expected output (approximately):
`ABORTED at peek 2: latency_ms harmed`
`  Z = 3.876, threshold = 3.226`
The experiment is aborted at peek 2 (50% of data) because the latency guardrail detected a significant increase. The primary KPI is never analysed — the experiment was stopped for safety.
If the latency harm were removed (e.g. `rng.normal(200, 50, N)` for treatment), the experiment would run to completion and the primary KPI result would be reported with a standard CI (no peeking penalty).

## A.6 Simulation: Comparing All Methods

This simulation runs all three methods (plus naive peeking) on the same experiments under the null, and reports their false positive rates.

```python
import numpy as np
from scipy import stats

def compare_methods(
    n_experiments: int = 10_000,
    n_obs_per_group: int = 2000,
    K: int = 4,
    alpha: float = 0.05,
    seed: int = 42,
) -> dict:
    """Compare false positive rates of all methods."""
    rng = np.random.default_rng(seed)
    peek_every = n_obs_per_group // K

    # Pre-compute critical values
    z_bonf = stats.norm.ppf(1 - alpha / (2 * K))
    c_P = pocock_critical_value(K, alpha)

    counts = {
        "naive": 0,
        "bonferroni": 0,
        "pocock": 0,
        "obf": 0,
    }

    for _ in range(n_experiments):
        ctrl = rng.normal(0, 1, n_obs_per_group)
        trt = rng.normal(0, 1, n_obs_per_group)

        rej = {m: False for m in counts}

        for k in range(1, K + 1):
            n = k * peek_every
            c, t = ctrl[:n], trt[:n]
            tau = t.mean() - c.mean()
```

```python
            se = np.sqrt(c.var(ddof=1)/n + t.var(ddof=1)/n)
            z = abs(tau / se)

            # Naive: standard z > 1.96
            if z > stats.norm.ppf(1 - alpha / 2):
                rej["naive"] = True

            # Bonferroni
            if z > z_bonf:
                rej["bonferroni"] = True

            # Pocock
            if z > c_P:
                rej["pocock"] = True

            # O'Brien-Fleming
            c_k = stats.norm.ppf(1 - alpha/2) / np.sqrt(k/K)
            if z > c_k:
                rej["obf"] = True

        for m, rejected in rej.items():
            if rejected:
                counts[m] += 1

    return {m: c / n_experiments for m, c in counts.items()}

# --- Run it ---
rates = compare_methods(K=4)
print(f"{'Method':<15} {'FPR':>8}")
print("-" * 25)
for method, fpr in rates.items():
    label = method.upper().replace("_", " ")
    marker = " ***" if fpr > 0.06 else ""
    print(f"{label:<15} {fpr:>7.1%}{marker}")
```

Expected output (approximately):

```
Method              FPR
-------------------------
NAIVE            14.2% ***
BONFERRONI        3.1%
POCOCK            4.5%
OBF               4.8%
```

All corrected methods stay at or below 5%. Naive peeking exceeds it by a wide margin, even with only $K = 4$ peeks.